SentinelWeb Security Report

Generated: 2026-08-31 18:53:43 UTC

Report range: Beginning to Present

Summary

Attack findings	WAF requests	Allowed	Blocked
286	76	196	248

Attack distribution

Attack type	Findings
Prompt Injection	103
SQL Injection	115
XSS	68

Risk findings and mitigation

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 3d77fbc3-ce50-4421-a72d-d0fc783d93c7; request ID: 856; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 100 (Critical), confidence 1.00, action blocked

Correlation: c4e9a0d9-59cb-48c8-bc19-64f5e20c8d53; request ID: 855; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: a4c65c07-b9ff-4927-b3db-55cc21343d6d; request ID: 854; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 96 (Critical), confidence 0.95, action blocked

Correlation: 21c349fb-fcbd-4e06-a836-c05eaf923973; request ID: 852; component: unspecified; method: ml

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 873caf35-2a2d-4225-9891-1131f6d07533; request ID: 851; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 86 (Critical), confidence 0.91, action blocked

Correlation: b2c8f2dd-88ce-46ac-8df4-ac07bb204248; request ID: 849; component: body.prompt; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 50 (Medium), confidence 0.75, action blocked

Correlation: 0b7e9c3a-88f2-4ccf-a05c-d8faea221029; request ID: 847; component: header.user-agent; method: ml

Mitigation: Validate and isolate untrusted input before processing.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 0b7e9c3a-88f2-4ccf-a05c-d8faea221029; request ID: 847; component: query.q; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Prompt Injection - risk 50 (Medium), confidence 0.75, action blocked

Correlation: dba69c66-a876-4c53-bd91-dfa21e728159; request ID: 846; component: header.user-agent; method: ml

Mitigation: Validate and isolate untrusted input before processing.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: dba69c66-a876-4c53-bd91-dfa21e728159; request ID: 846; component: query.username; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 4542ae04-b071-4e6a-880b-5f11951fc80e; request ID: 844; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 2a3033b4-c54a-4177-84be-b473450959c3; request ID: 843; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: fef10a4b-d60d-409a-9ee1-70e0690e8af6; request ID: 830; component: query.value; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: ce7ee258-e918-42a6-bec5-bcedf9ff1a91; request ID: 829; component: body.value; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 6d13dd9c-8941-484b-bc5d-29b5ca8c2ef1; request ID: 828; component: body.value; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: 4aa2590b-5b89-4bec-bba4-9148f9bf0780; request ID: 814; component: query.value; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: ad2bef64-76c2-4a92-a2d2-7494aee74af1; request ID: 813; component: body.value; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: d59afe4f-91e6-4035-9eee-befda02be1f1; request ID: 812; component: body.value; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 48 (Medium), confidence 0.72, action blocked

Correlation: 08c9c9ef-9fff-416e-bf14-3c3f748d73f0; request ID: 801; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 48 (Medium), confidence 0.72, action blocked

Correlation: 751dddee-0cd2-448e-b47c-2b42a5670113; request ID: 800; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 48 (Medium), confidence 0.72, action blocked

Correlation: 1b11fe62-c516-480d-a334-0435ac353990; request ID: 799; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 48 (Medium), confidence 0.72, action blocked

Correlation: 4ef07d49-012b-45a1-bd04-9af5d0cec8de; request ID: 798; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 48 (Medium), confidence 0.72, action blocked

Correlation: 4dd75faf-f7cd-402f-b80c-8729b4950feb; request ID: 797; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 48 (Medium), confidence 0.72, action blocked

Correlation: 542050b8-4001-456c-b93c-59f093c7e6b1; request ID: 796; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: a38fcb5f-c5c2-46d5-aca3-603ad8621be4; request ID: 793; component: body.prompt; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 317eb047-421e-4e19-876c-e125737df0d0; request ID: 792; component: query.q; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 44e23beb-2ae2-4db1-bb62-df0da3f7da9d; request ID: 791; component: query.username; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 48 (Medium), confidence 0.72, action allowed

Correlation: 36f4e4c6-5d39-4191-8923-4c69be3a7dab; request ID: 790; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 48 (Medium), confidence 0.72, action allowed

Correlation: 3868c78a-2842-464f-bdd1-ef968b99b84c; request ID: 789; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 48 (Medium), confidence 0.72, action allowed

Correlation: d79a0733-edaf-4da3-9420-6b8b72f5daaa; request ID: 788; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 50 (Medium), confidence 0.75, action allowed

Correlation: 443a6eeb-e5be-4c30-b942-268cac824876; request ID: 787; component: query.message; method: ml

Mitigation: Validate and isolate untrusted input before processing.

XSS - risk 46 (Medium), confidence 0.69, action allowed

Correlation: 443a6eeb-e5be-4c30-b942-268cac824876; request ID: 787; component: query.upstream_url; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 48 (Medium), confidence 0.72, action allowed

Correlation: 443a6eeb-e5be-4c30-b942-268cac824876; request ID: 787; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: ef15c30d-3fc4-41f3-851a-7ec4cb78af8b; request ID: 786; component: body.prompt; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 0970fa95-03a4-4c6b-8828-887f694bf4c4; request ID: 785; component: query.q; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: c75db475-88c4-4930-9018-1ef8ec3154f1; request ID: 784; component: query.username; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 46 (Medium), confidence 0.69, action allowed

Correlation: 835ee7d2-a772-4004-a654-f462d5573ffb; request ID: 781; component: body.message; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 48 (Medium), confidence 0.72, action allowed

Correlation: 835ee7d2-a772-4004-a654-f462d5573ffb; request ID: 781; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 48 (Medium), confidence 0.72, action allowed

Correlation: 443a6eeb-e5be-4c30-b942-268cac824876; request ID: 780; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 8426b77d-9193-4685-9cc1-09805af59012; request ID: 777; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 42ff26a1-dc42-42c2-ae7b-3e99409bfa39; request ID: 776; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-408; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 32f7120a-fee8-495c-9402-26ec97698adf; request ID: 775; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 32f7120a-fee8-495c-9402-26ec97698adf; request ID: 775; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 32f7120a-fee8-495c-9402-26ec97698adf; request ID: 775; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 32f7120a-fee8-495c-9402-26ec97698adf; request ID: 775; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 9e5543cb-55f0-40bf-b9c7-e847aec964d0; request ID: 774; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 5ffec2e4-479d-444e-bc19-e440593b9dee; request ID: 773; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 3863691c-8262-483b-88bf-55d5f568dce6; request ID: 772; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 3b87255b-70ea-483e-aa22-49fe3888a2d2; request ID: 771; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: f5e29ebf-dd80-4155-b7fd-a41a608026d3; request ID: 770; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: dbcc8cfc-8de6-4d6c-bb11-67cfacd4cafb; request ID: 769; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 6e43fa4a-6fe4-4241-a903-56792103e546; request ID: 768; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: d1ba36f4-9aaa-43ec-a82e-93b890da0e14; request ID: 767; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 446a4451-da04-4436-ac90-857b734f6d1c; request ID: 757; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 30ffe8ee-4960-4aa4-9d4f-506ddb6aaf17; request ID: 756; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 42bc6077-3de3-449c-a69e-f908e6816415; request ID: 755; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: 7080056b-6b48-4550-8725-99e534411e00; request ID: 754; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 2ac8d005-22eb-4ee8-80ed-19fae0b2893e; request ID: 753; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 61293637-7cb9-459a-9f94-3b56c2aec2e9; request ID: 752; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 525b4c63-8e2f-40e4-86ed-e35084a39754; request ID: 751; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: e7dbf265-91f8-4da3-88a3-727f80477a80; request ID: 750; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: 4eb43151-696a-48e7-8461-7d8cc0af6c97; request ID: 740; component: body.prompt; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: ca077ed8-ad14-4f08-9756-83f760b768fb; request ID: 739; component: query.q; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: b02fcb7e-cc55-4963-ad09-76f5d3b5e65a; request ID: 738; component: query.username; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 48 (Medium), confidence 0.72, action allowed

Correlation: ac59cce0-cd4e-4d04-a93f-8df61e9eb107; request ID: 737; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: edecb797-f4ea-43d4-99bb-8988c3087ea1; request ID: 736; component: body.prompt; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: bf7b93ae-d138-4f54-8e7d-909368816910; request ID: 735; component: query.q; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 9becdd94-1a04-437c-b2f6-3d19f21da748; request ID: 734; component: query.username; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 68 (High), confidence 0.72, action allowed

Correlation: e505a264-f27f-4ab2-a9dc-fcd43658e09b; request ID: 733; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 187bc167-d57c-4711-88ab-44cfe760ee93; request ID: 730; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 16e57cfd-cb7f-439c-9a18-9b86a66502db; request ID: 729; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-377; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 69fb122a-cf88-4c43-80ec-41601d425362; request ID: 728; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 69fb122a-cf88-4c43-80ec-41601d425362; request ID: 728; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 69fb122a-cf88-4c43-80ec-41601d425362; request ID: 728; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 69fb122a-cf88-4c43-80ec-41601d425362; request ID: 728; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 241e532d-913d-4668-9548-0cf398403c41; request ID: 727; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 7607307d-95c9-49a4-9842-f80e7f97e649; request ID: 726; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 787d6609-f1b6-4171-a524-0477759a3b84; request ID: 725; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 856c0820-d96a-4fb7-9ac1-7824f0c43f86; request ID: 724; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: 2fdadb7c-aac8-4277-aed1-28d89d9442f1; request ID: 723; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: 35b6d15a-127f-44dc-8ab3-da8a89400af3; request ID: 722; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: e644d7e3-7e0c-455f-88d3-f797d84fb7e6; request ID: 721; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: cc429d2e-892e-42b2-bd10-1ee719bc0490; request ID: 720; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: f34dec08-f957-44dd-b2dd-649a3c8b565b; request ID: 710; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 06b1ec03-a4df-4aeb-91ca-547360dd9f82; request ID: 709; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: b6d2eaef-4812-47bc-bd49-def9bcd0e5be; request ID: 708; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: dd899080-0960-448d-994c-1de16a286ec0; request ID: 707; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: b124094a-c44b-4073-b80f-784083a4d5dc; request ID: 706; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 8a3b9d6e-fd73-4fa7-9e7b-e18157b86667; request ID: 705; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 97034829-da67-4904-ac5f-7f2bd950f4a6; request ID: 704; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: e7bb1661-6a15-4720-bd1b-db94931ecf0e; request ID: 703; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: 1790c45d-37fd-4d15-bb86-a89b0e71ea96; request ID: 693; component: body.prompt; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 6909bedb-857c-4064-8402-46ea3ff85025; request ID: 692; component: query.q; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 5f514593-958e-44bd-ad8c-93cfad12ca53; request ID: 691; component: query.username; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 68 (High), confidence 0.72, action allowed

Correlation: 7951630c-8ebb-43d1-91ef-b6ff4b4d2fef; request ID: 690; component: path; method: ml

Mitigation: Validate and isolate untrusted input before processing.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: bf7c42bf-f7a4-4810-b5a3-070107fa544c; request ID: 687; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 282511c6-cd73-4f5b-be28-b7ad766c5211; request ID: 686; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-350; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: d7b1ae00-c103-4724-91d8-bfb51afc98fe; request ID: 685; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: d7b1ae00-c103-4724-91d8-bfb51afc98fe; request ID: 685; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: d7b1ae00-c103-4724-91d8-bfb51afc98fe; request ID: 685; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: d7b1ae00-c103-4724-91d8-bfb51afc98fe; request ID: 685; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 8b77ae03-6f71-486e-9e75-ea518486ae43; request ID: 684; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: d187d96f-1d1d-4e6f-9a9f-2545488c99f7; request ID: 683; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 2c89c3e5-ca8c-42c5-8b80-92a5d3fbae4f; request ID: 682; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 71b5053b-d6b5-4f96-94ff-5dc94d90600a; request ID: 681; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: 06640667-710c-49d2-9b15-c168e806d82e; request ID: 680; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: fab4ed93-64c9-4aa1-b73b-b7633955419f; request ID: 679; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: c8e92a97-374b-497c-880e-5adaa1e62f2c; request ID: 678; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: b3c00170-abdf-46ab-bc26-c5370dafb919; request ID: 677; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: bedc0486-c487-4a11-8ab9-3d9ed7735cba; request ID: 667; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 02fe3de3-4403-4c73-8cc3-2dee6ce2dec9; request ID: 666; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: bd046771-e8f8-4d32-8fe1-0522a5db9f33; request ID: 665; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: 64d5e046-ebda-4904-b877-33f007a1cd92; request ID: 664; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 8e36c608-1795-4991-bd55-075a419e165d; request ID: 663; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: b1f7a66d-9786-45b8-a4c6-fec540c0c55e; request ID: 662; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 001569d6-3153-4135-ad0d-d066dc9d97bc; request ID: 661; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 01382abb-be81-4571-87ae-8f2afc2f8a00; request ID: 660; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 6ff230bf-5b6b-4f22-9d90-2fc9edca7d15; request ID: 648; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 568792ae-cf8b-4010-8d33-481bb985d99a; request ID: 647; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-327; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 4d80607e-e586-49f2-a9a0-7df072e25115; request ID: 646; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 4d80607e-e586-49f2-a9a0-7df072e25115; request ID: 646; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 4d80607e-e586-49f2-a9a0-7df072e25115; request ID: 646; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 4d80607e-e586-49f2-a9a0-7df072e25115; request ID: 646; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 4ed38870-1283-4b85-840a-eb9dd84523d7; request ID: 645; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 34e283a3-9572-431e-be7f-e09f0e416f5c; request ID: 644; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 028d7282-c60e-4817-b70a-39186dd8ddf7; request ID: 643; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: edae440f-acc4-4689-b15d-cb71876aa4a3; request ID: 642; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: 4d76899d-6e82-4efd-ad66-2c4e8f5f5603; request ID: 641; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: becf32a0-7938-4d50-9727-f6a3ae7da2c2; request ID: 640; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 8d586b3c-abc6-4b0a-8a66-f9f91e32bfe0; request ID: 639; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: f4de3df2-e73f-4c5a-808d-b7d156898c5b; request ID: 638; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 6d9d5839-8389-4e2e-978a-be2e0edc4ca7; request ID: 628; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 3e50e84d-588c-4665-92c3-77dafe0b660c; request ID: 627; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 8da6c65a-9da5-4c12-a92b-730f5c3217b6; request ID: 626; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: 0c910724-7ab4-437c-a7c5-5c32d588f993; request ID: 625; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: a7b85acb-e9fa-498d-890e-f4b1745e9dcc; request ID: 624; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: c88d8375-6262-4612-b399-31a4a70cd11c; request ID: 623; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: a9289722-9e91-4d5d-8eb0-2ca6f75e9a42; request ID: 622; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: da91a690-30de-4ea7-885c-e8fbd0048ed9; request ID: 621; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 5a26c966-d7bf-44eb-95de-594a28fca5dc; request ID: 609; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 760994c2-86ec-4cd6-9a9e-0d9ba2d227be; request ID: 608; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-304; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: dc6925dd-de1a-46c1-b0c7-d7dd72279139; request ID: 607; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: dc6925dd-de1a-46c1-b0c7-d7dd72279139; request ID: 607; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: dc6925dd-de1a-46c1-b0c7-d7dd72279139; request ID: 607; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: dc6925dd-de1a-46c1-b0c7-d7dd72279139; request ID: 607; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 2cafac88-fcd1-4672-9a3a-4d56f57ed356; request ID: 606; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 1ea5c6b9-3747-43a8-b328-508a1496e9e1; request ID: 605; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 10a2fec3-b87f-4960-a672-c7a977787f5a; request ID: 604; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: f8b04554-c5a3-44f5-bf20-1da6fd3df7d9; request ID: 603; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: 49a9092d-3282-4b36-a2ac-909a20cbf064; request ID: 602; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: 8dd300e7-2435-4203-8c96-48402353e238; request ID: 601; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 83964280-4aa5-44bb-a5ad-3d68bb76e19b; request ID: 600; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 6d79e8f6-03cb-4c87-b53a-7beb35d38029; request ID: 599; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: f0f09ae6-cce8-4986-a509-b13a15625ddc; request ID: 589; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: c6261b93-191c-49c9-8bac-98ed52b0d17b; request ID: 588; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 5f1b59bd-1c3d-47bc-8ab2-2c373d0532a2; request ID: 587; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: fc9a9bd9-08b6-44dd-b24b-ccb9681e7d72; request ID: 586; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: c90a8ac8-a6e0-44b2-982c-693d4ea682c7; request ID: 585; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 9c2d0d22-0fe6-499c-8d34-2763565bb2b1; request ID: 584; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 7f7f8853-f21a-4169-be92-81a6a82f58ba; request ID: 583; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 75b99aa0-77e1-4249-ba7c-987d47115e9e; request ID: 582; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: be46df1e-089b-469f-b3ee-5e8d492114eb; request ID: 569; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: ac6dc951-377b-4bce-b713-0a5e46610b81; request ID: 568; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-281; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 1839591e-c138-4bdc-b906-ff8c183d4000; request ID: 567; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 1839591e-c138-4bdc-b906-ff8c183d4000; request ID: 567; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 1839591e-c138-4bdc-b906-ff8c183d4000; request ID: 567; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 1839591e-c138-4bdc-b906-ff8c183d4000; request ID: 567; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 5da3b7c4-46a4-4ab8-ba89-465b092a8acc; request ID: 566; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 595e06b5-85ea-4f90-8579-b6a01d019608; request ID: 565; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 448f5afa-deaa-49f1-9682-ad60b4570d33; request ID: 564; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 86d34da0-8b87-4408-b923-6b60343376af; request ID: 563; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: acdc8a30-6e4f-4120-8f03-1883332bf587; request ID: 562; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: db47c2bc-72a0-45b3-9ef4-0e0b474ddfa0; request ID: 561; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 1a0071c9-b2bd-4da9-937e-f2ef9a41d075; request ID: 560; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 2edae09d-181d-435d-b844-b9dec9565708; request ID: 559; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: d1057dc6-82ad-4187-b154-0816fbf0288c; request ID: 549; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: 3636754a-f4bd-41af-930e-2254a29c8187; request ID: 548; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: c26469ae-0672-435f-a6ea-bf9f1a7cd19b; request ID: 547; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: cb342ba2-7d8f-48d4-b629-8f5289c003a7; request ID: 546; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: b60c2159-477d-413c-9908-275417151896; request ID: 545; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: ffc8612f-8c05-40e1-a317-b6f63382c1a8; request ID: 544; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 95ef6d39-cea7-4ab6-8349-1659859e3f41; request ID: 543; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: 9e30fa9d-3124-4ff2-9f30-d11851f1022c; request ID: 542; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-260; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-259; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-258; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-257; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-256; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-255; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-254; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-253; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-252; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-251; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-250; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: finding-249; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: finding-248; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-247; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-246; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-245; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-244; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-243; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: finding-242; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-241; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-240; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-239; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-238; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-237; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-236; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-235; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-234; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-233; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-232; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-231; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-230; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-229; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-228; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-227; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: finding-226; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: finding-225; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-224; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-223; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-222; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-221; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-220; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: finding-219; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-218; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-217; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-216; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-215; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-214; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-213; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-212; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-211; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-210; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-209; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-208; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-207; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-206; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-205; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-204; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: finding-203; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: finding-202; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-201; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-200; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-199; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-198; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-197; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: finding-196; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-195; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-194; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-193; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-192; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-191; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-190; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-189; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-188; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-187; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-186; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-185; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-184; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-183; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-182; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-181; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: finding-180; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: finding-179; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-178; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-177; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-176; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-175; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-174; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: finding-173; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-172; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-171; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-170; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-169; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-168; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-167; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-166; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-165; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-164; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-163; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-162; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-161; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-160; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-159; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-158; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: finding-157; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: finding-156; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-155; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-154; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-153; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-152; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-151; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: finding-150; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-149; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-148; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-147; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-146; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-145; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-144; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-143; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-142; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-141; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-140; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-139; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-138; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-137; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: finding-136; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: finding-135; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-134; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-133; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-132; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-131; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-130; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: finding-129; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-128; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-127; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-126; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-125; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-124; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-123; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-122; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-121; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-120; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-119; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-118; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-117; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-116; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: finding-115; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: finding-114; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-113; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-112; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-111; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-110; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-109; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: finding-108; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-107; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-106; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-105; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-104; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 57 (Medium), confidence 0.85, action allowed

Correlation: finding-103; request ID: unavailable; component: unspecified; method: ml

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-102; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-101; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-100; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-99; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-98; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-97; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-96; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: finding-95; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: finding-94; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-93; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-92; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-91; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-90; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-89; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Review and validate the affected input.

Prompt Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: finding-88; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-87; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-86; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-85; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-84; request ID: unavailable; component: unspecified; method: hybrid

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-83; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-82; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-81; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: finding-80; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-79; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-78; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-77; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-76; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-75; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-74; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-73; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-72; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: finding-71; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-70; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-69; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-68; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-67; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-66; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-65; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-64; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

XSS - risk 91 (Critical), confidence 0.96, action blocked

Correlation: finding-63; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Prompt Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: finding-62; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 86 (Critical), confidence 0.90, action blocked

Correlation: finding-61; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

SQL Injection - risk 93 (Critical), confidence 0.98, action blocked

Correlation: finding-60; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 93 (Critical), confidence 0.98, action blocked

Correlation: finding-59; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 89 (Critical), confidence 0.94, action blocked

Correlation: finding-58; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-57; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-56; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-55; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-54; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-53; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-52; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-51; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

XSS - risk 68 (High), confidence 0.75, action allowed

Correlation: finding-50; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Prompt Injection - risk 83 (Critical), confidence 0.92, action blocked

Correlation: finding-49; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 76 (High), confidence 0.85, action allowed

Correlation: finding-48; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

SQL Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: finding-47; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 91 (Critical), confidence 0.96, action blocked

Correlation: finding-46; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 87 (Critical), confidence 0.92, action blocked

Correlation: finding-45; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 90 (Critical), confidence 1.00, action blocked

Correlation: finding-44; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 90 (Critical), confidence 1.00, action blocked

Correlation: finding-43; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-42; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-41; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-40; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-39; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-38; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-37; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-36; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Prompt Injection - risk 83 (Critical), confidence 0.92, action blocked

Correlation: finding-35; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

Prompt Injection - risk 90 (Critical), confidence 1.00, action blocked

Correlation: finding-34; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 90 (Critical), confidence 1.00, action blocked

Correlation: finding-33; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-32; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

SQL Injection - risk 95 (Critical), confidence 1.00, action blocked

Correlation: finding-31; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-30; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Prompt Injection - risk 50 (Medium), confidence 0.67, action allowed

Correlation: finding-29; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 85 (Critical), confidence 1.00, action blocked

Correlation: finding-28; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 56 (Medium), confidence 0.60, action allowed

Correlation: finding-27; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-26; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Prompt Injection - risk 50 (Medium), confidence 0.67, action allowed

Correlation: finding-25; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 85 (Critical), confidence 1.00, action blocked

Correlation: finding-24; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 56 (Medium), confidence 0.60, action allowed

Correlation: finding-23; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Prompt Injection - risk 75 (High), confidence 1.00, action allowed

Correlation: finding-22; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 85 (Critical), confidence 1.00, action blocked

Correlation: finding-21; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 56 (Medium), confidence 0.60, action allowed

Correlation: finding-20; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-19; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

XSS - risk 85 (Critical), confidence 1.00, action blocked

Correlation: finding-18; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-17; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Prompt Injection - risk 50 (Medium), confidence 0.67, action allowed

Correlation: finding-16; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 85 (Critical), confidence 1.00, action blocked

Correlation: finding-15; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 56 (Medium), confidence 0.60, action allowed

Correlation: finding-14; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

XSS - risk 63 (High), confidence 0.75, action allowed

Correlation: finding-13; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

XSS - risk 63 (High), confidence 0.75, action allowed

Correlation: finding-12; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-11; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Prompt Injection - risk 50 (Medium), confidence 0.67, action allowed

Correlation: finding-10; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 85 (Critical), confidence 1.00, action blocked

Correlation: finding-9; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 56 (Medium), confidence 0.60, action allowed

Correlation: finding-8; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-7; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

Prompt Injection - risk 50 (Medium), confidence 0.67, action allowed

Correlation: finding-6; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Never pass raw user input directly to LLM system prompts. Use input/output filtering and sandwich defense patterns. Implement role-based prompt isolation.

XSS - risk 85 (Critical), confidence 1.00, action blocked

Correlation: finding-5; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 56 (Medium), confidence 0.60, action allowed

Correlation: finding-4; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.

Security finding - risk 0 (Safe), confidence 0.00, action allowed

Correlation: finding-3; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Review and validate the affected input.

XSS - risk 85 (Critical), confidence 1.00, action blocked

Correlation: finding-2; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Encode all user-supplied output (HTML entity encoding). Use Content-Security-Policy headers. Sanitize inputs using a library like DOMPurify on the client side.

SQL Injection - risk 56 (Medium), confidence 0.60, action allowed

Correlation: finding-1; request ID: unavailable; component: unspecified; method: rule_based

Mitigation: Use parameterized queries (prepared statements). Validate and sanitize all user inputs. Apply the principle of least privilege to database accounts.